Divvy Bike-Share
Exploratory Data Analysis Report

April 2020 · Chicago, IL

84,717	61,112	23,605	73.1 min
Total Rides	Members	Casual Riders	Avg Casual Ride

1. Introduction & Dataset Overview

This report presents an exploratory data analysis (EDA) of the Divvy bike-share trip data for April 2020 in Chicago, Illinois. The analysis focuses on understanding the behavioral differences between **annual members** and **casual (pay-per-ride) users**, with the goal of informing business strategy and marketing decisions.

Dataset Summary

Attribute	Value
File	202004-divvy-tripdata.csv
Total records (raw)	84,775
Records after cleaning	84,717
Columns	14
Date range	April 1 – April 30, 2020
Bike type	docked_bike (single type)

Notebook Setup & Data Load (from source notebook)

```
In [1]: import pandas as pd
        import numpy as np
        import seaborn as sns
        import matplotlib.pyplot as plt
```

```
In [2]: df=pd.read_csv("202004-divvy-tripdata.csv")
```

```
In [10]: df.head()
```

Out[10]:

	ride_id	rideable_type	started_at	ended_at	start_station_name	start_station
0	A847FADBBC638E45	docked_bike	2020-04-26 17:45:14	2020-04-26 18:12:03	Eckhart Park	
1	5405B80E996FF60D	docked_bike	2020-04-17 17:08:54	2020-04-17 17:17:03	Drake Ave & Fullerton Ave	5
2	5DD24A79A4E006F4	docked_bike	2020-04-01 17:54:13	2020-04-01 18:08:36	McClurg Ct & Erie St	1
3	2A59BBDF5CDBA725	docked_bike	2020-04-07 12:50:19	2020-04-07 13:02:31	California Ave & Division St	2
4	27AD306C119C6158	docked_bike	2020-04-18 10:22:59	2020-04-18 11:15:54	Rush St & Hubbard St	1

```
In [11]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 84717 entries, 0 to 84775
Data columns (total 14 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   ride_id             84717 non-null  object
 1   rideable_type       84717 non-null  object
 2   started_at          84717 non-null  datetime64[ns]
 3   ended_at            84717 non-null  datetime64[ns]
 4   start_station_name  84717 non-null  object
 5   start_station_id    84717 non-null  int64
 6   end_station_name    84620 non-null  object
 7   end_station_id      84620 non-null  float64
 8   start_lat           84717 non-null  float64
 9   start_lng           84717 non-null  float64
 10  end_lat             84620 non-null  float64
 11  end_lng             84620 non-null  float64
 12  member_casual       84717 non-null  object
 13  ride_length         84717 non-null  float64
dtypes: datetime64[ns](2), float64(6), int64(1), object(5)
memory usage: 9.7+ MB
```

```
In [12]: df.describe()
```

Figure 1 – Jupyter notebook: imports, data load, and df.info() output.

2. Data Cleaning & Feature Engineering

Before analysis, several pre-processing steps were applied to ensure data quality and to create derived features useful for temporal analysis.

- **Datetime conversion** – *started_at* and *ended_at* were cast to pandas *datetime64* to enable time arithmetic.
- **Ride length** – calculated as (ended_at – started_at) in minutes; stored as *ride_length*.
- **Invalid ride removal** – 58 records with ride_length ≤ 0 were dropped, leaving 84,717 valid rows.
- **Day of week** – extracted as a named string column (Monday…Sunday).
- **Month** – extracted as month name string.
- **Hour** – extracted as an integer 0–23 for hourly analysis.

Out[12]:

	started_at	ended_at	start_station_id	end_station_id	start_lat	8
count	84717	84717	84717.000000	84620.000000	84717.000000	
mean	2020-04-15 22:36:46.175726336	2020-04-15 23:12:38.685800960	236.688717	236.979284	41.908154	
min	2020-04-01 00:00:30	2020-04-01 00:10:45	2.000000	2.000000	41.736600	
25%	2020-04-07 22:24:02	2020-04-07 23:24:53	113.000000	113.000000	41.884600	
50%	2020-04-16 15:21:22	2020-04-16 15:46:16	211.000000	212.000000	41.903000	
75%	2020-04-22 19:38:48	2020-04-22 20:14:41	324.000000	323.000000	41.932600	
max	2020-04-30 23:57:20	2020-05-30 15:30:55	673.000000	673.000000	42.064900	
std	NaN	NaN	160.273682	160.944014	0.043317	

Data Cleaning

1. convertibg to date time

In [13]:
```
df['started_at']=pd.to_datetime(df['started_at'])
```

In [14]:
```
df['ended_at']=pd.to_datetime(df['ended_at'])
```

2.Creating Ride Length

In [15]:
```
df['ride_length']=(df['ended_at']-df['started_at']).dt.total_seconds()/60
```

3.Removing invalid rides

In [16]:
```
df=df[df['ride_length']>0]
```

Creating Some New Columns

In [17]:
```
df['day_of_week']=df['started_at'].dt.day_name()
```

In [18]:
```
df['month'] = df['started_at'].dt.month_name()
```

In [19]:
```
df['hour'] = df['started_at'].dt.hour
```

Verification

In [20]:
```
df.head()
```

Figure 2 – df.describe() summary statistics and cleaning code.

3. Exploratory Data Analysis

3.1 User Type Distribution

The dataset contains rides from two user categories. Annual members account for the majority of rides (72.1%), while casual users contribute 27.9%. This suggests the service is well-adopted by regular commuters.

User Type	Ride Count	Share (%)
Member	61,112	72.1%
Casual	23,605	27.9%
Total	**84,717**	**100%**

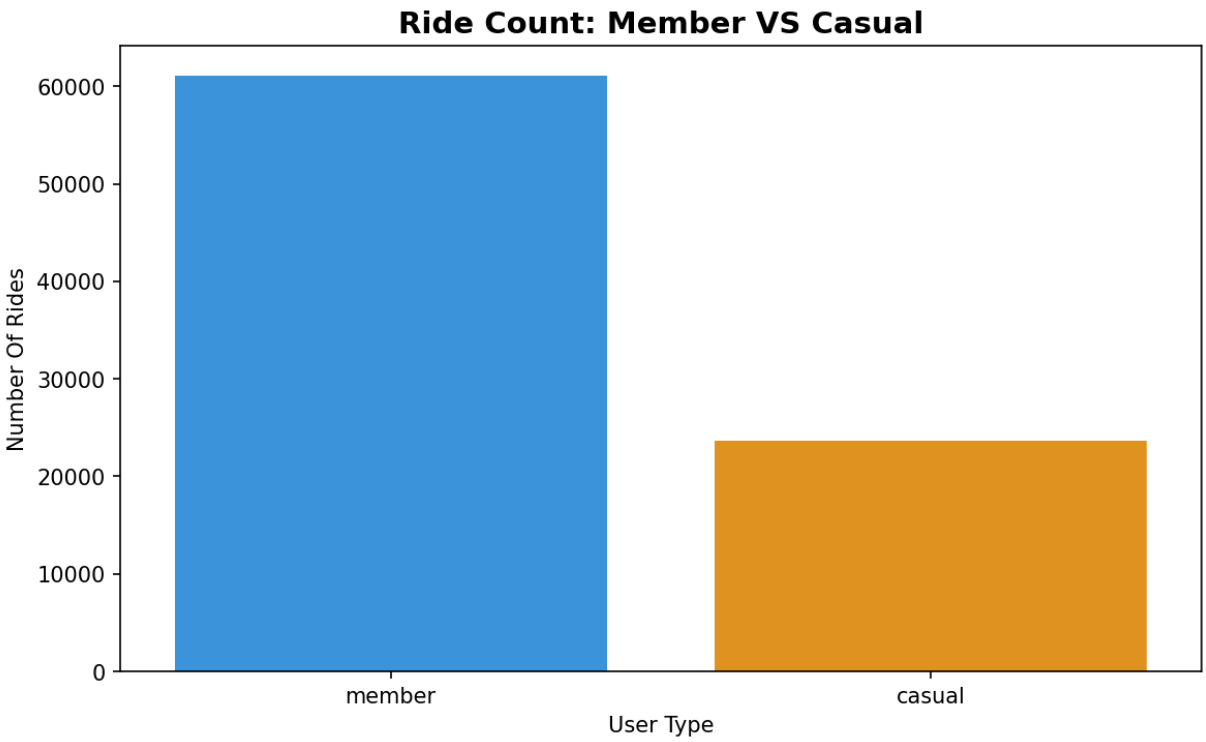


Figure 3 – Ride count comparison: Member vs Casual riders.

3.2 Average Ride Length by User Type

Casual riders take significantly longer rides on average (73.1 minutes) compared to members (21.5 minutes) — a difference of 3.4×. This strongly indicates that casual riders use the bikes for leisure or tourism, while members prefer shorter, goal-oriented commute trips.

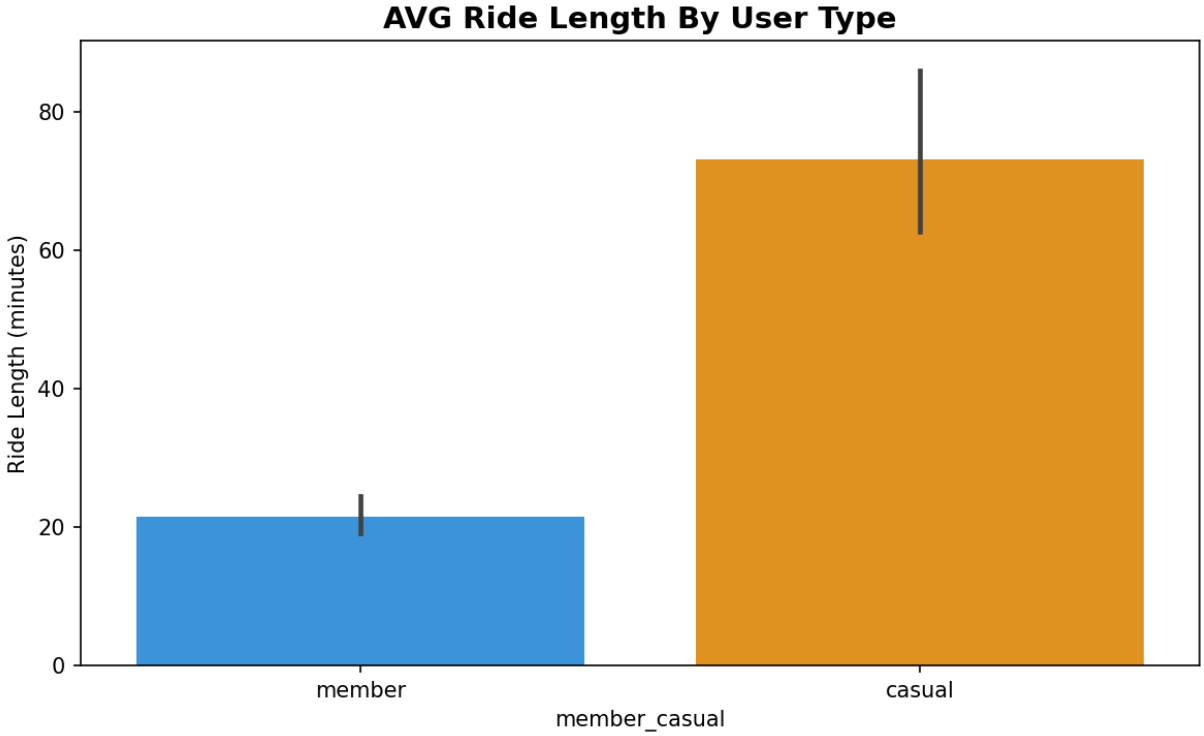


Figure 4 – Average ride length (minutes) by user type.

3.3 Peak Hours Analysis

The hourly pattern reveals distinctly different usage profiles for each user group:

- **Early morning (0–5 AM)** – Very low activity across both groups.
- **Morning peak (6–9 AM)** – Strong member surge (commute pattern); casuals remain low.
- **Midday (10 AM–3 PM)** – Casual ridership grows steadily; members stay elevated.
- **Evening peak (4–6 PM)** – Largest spike for members (evening commute); casuals also rise.
- **Night (7–11 PM)** – Both groups decline; members drop sharply.

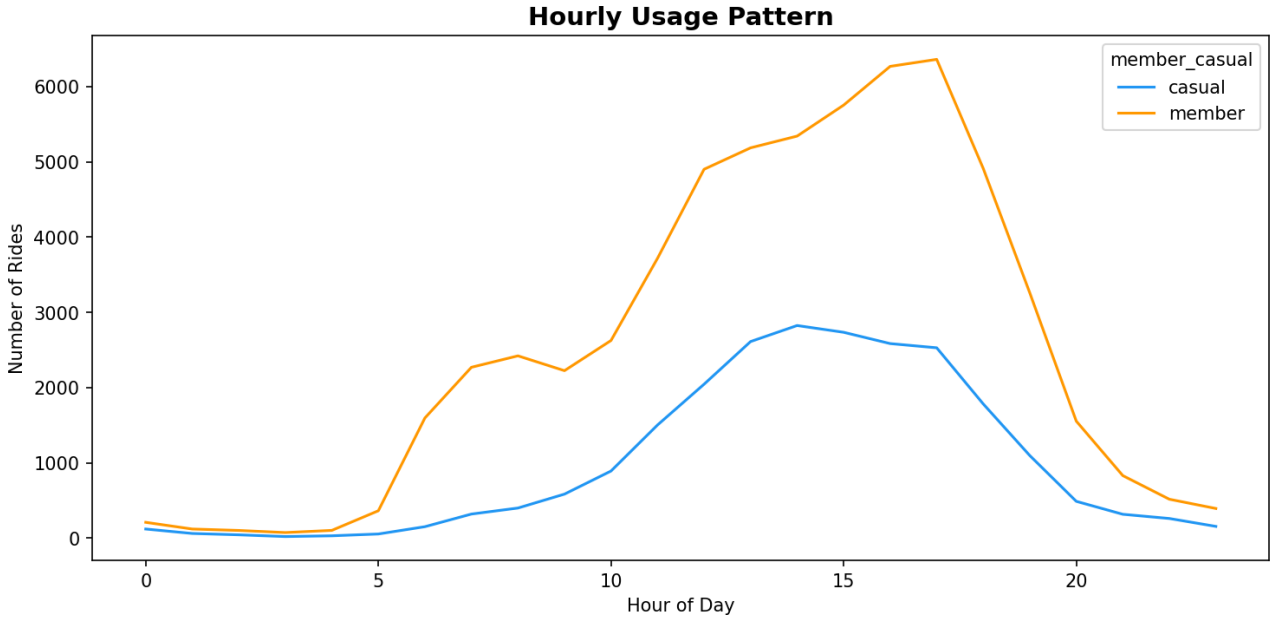


Figure 5 – Hourly usage pattern: members peak at commute hours; casuals peak midday.

3.4 Day-of-Week Ridership

Sunday records the highest ridership for both user types. Members maintain relatively consistent weekday usage (commute behaviour), while casual ridership spikes on weekends, further confirming a leisure-driven motivation for this group.

Day	Casual	Member	Total
Monday	2,673	8,059	10,732
Tuesday	3,654	9,148	12,802
Wednesday	1,795	6,912	8,707
Thursday	2,439	9,263	11,702
Friday	2,508	7,458	9,966
Saturday	4,065	8,839	12,904
Sunday	6,471	11,433	17,904

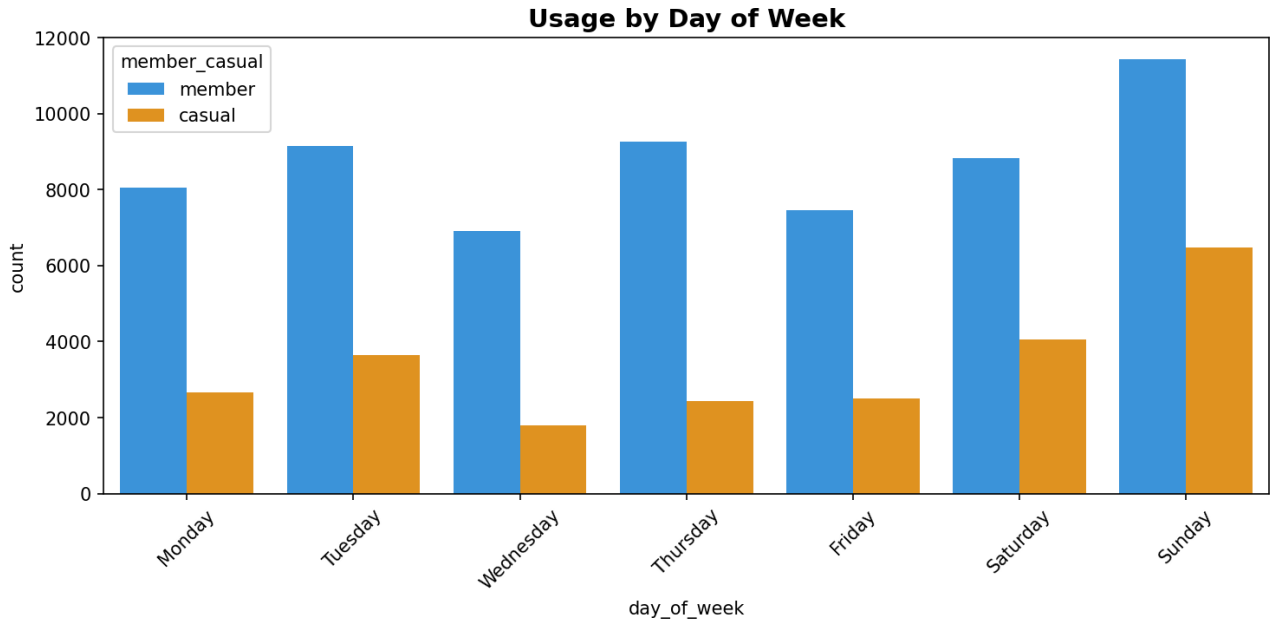


Figure 6 – Day-of-week usage: casual riders spike on weekends.

4. Key Insights & Recommendations

Members dominate volume	61,112 member rides vs 23,605 casual — members are the core revenue driver and should be retained through loyalty benefits.
Casual riders ride 3.4× longer	Average casual ride (73.1 min) vs member (21.5 min) suggests leisure use. Weekend promotions could increase casual frequency.
Commute vs leisure split	Member peaks at 7–9 AM and 4–6 PM align with work commute. Casual peaks at 10 AM–3 PM point to sightseeing/leisure activity.
Sunday is peak day	Both groups peak on Sunday. Weekend campaigns targeting casual riders could drive membership conversions.
Single bike type	All rides in April 2020 used docked_bike — no opportunity to differentiate by vehicle class in this dataset period.

5. Source Notebook Screenshots

The following pages reproduce the original Jupyter notebook for reference.

Out[20]:

	ride_id	rideable_type	started_at	ended_at	start_station_name	start_station
0	A847FADBBC638E45	docked_bike	2020-04-26 17:45:14	2020-04-26 18:12:03	Eckhart Park	
1	5405B80E996FF60D	docked_bike	2020-04-17 17:08:54	2020-04-17 17:17:03	Drake Ave & Fullerton Ave	5
2	5DD24A79A4E006F4	docked_bike	2020-04-01 17:54:13	2020-04-01 18:08:36	McClurg Ct & Erie St	1
3	2A59BBDF5CDBA725	docked_bike	2020-04-07 12:50:19	2020-04-07 13:02:31	California Ave & Division St	2
4	27AD306C119C6158	docked_bike	2020-04-18 10:22:59	2020-04-18 11:15:54	Rush St & Hubbard St	1

In [21]:
```
df[['ride_length','day_of_week','month','hour']].head()
```

Out[21]:

	ride_length	day_of_week	month	hour
0	26.816667	Sunday	April	17
1	8.150000	Friday	April	17
2	14.383333	Wednesday	April	17
3	12.200000	Tuesday	April	12
4	52.916667	Saturday	April	10

Lets Start with EDA

Total Rides

In [22]:
```
df['member_casual'].value_counts()
```

Out[22]:
```
member_casual
member    61112
casual    23605
Name: count, dtype: int64
```

Here we can observe that members use bikes more as compared to casuals (non-members)

AVG Ride Length

In [23]:
```
df.groupby('member_casual')['ride_length'].mean()
```

Out[23]:
```
member_casual
casual    73.142555
member    21.480341
Name: ride_length, dtype: float64
```

Here we can observe that casual riders usually ride longer

Ride By Day of Week (Weekend VS Weekday behavior)

In [24]:
```
df.groupby(['day_of_week','member_casual']).size().unstack()
```

Figure 7 – Notebook page 3.

Out[24]:

member_casual	casual	member
day_of_week		
Friday	2508	7458
Monday	2673	8059
Saturday	4065	8839
Sunday	6471	11433
Thursday	2439	9263
Tuesday	3654	9148
Wednesday	1795	6912

Here we can observe that Sundys have higher bike rides

Peak Hours Analysis

In [25]:
```
df.groupby(['hour','member_casual']).size().unstack()
```

Out[25]:

member_casual	casual	member
hour		
0	120	209
1	61	120
2	43	101
3	20	73
4	30	102
5	54	363
6	151	1596
7	319	2270
8	399	2422
9	584	2225
10	891	2626
11	1503	3719
12	2045	4902
13	2612	5187
14	2825	5343
15	2735	5755
16	2585	6270
17	2529	6363
18	1783	4913
19	1097	3260
20	488	1554
21	316	830
22	260	516
23	155	393

Figure 8 – Notebook page 4.

Here we can observe that : .Early Morning 0–5 AM: Very low activity (no casuals,low members). .Morning Peak (6–9 AM): 6 AM → jump starts 7–9 AM → strong growth Members >> Casual. .Midday (10 AM – 3 PM) : Both users increase, but: Casual riders grow steadily Members remain high. .Evening Peak (4 PM – 6 PM): BIGGEST spike in members Casual also increases but less. .Night (7 PM – 11 PM): Decline starts: Members drop sharply Casual still active slightly.

Bike Type Preferance

In [26]:
```
df.groupby(['rideable_type','member_casual']).size().unstack()
```

Out[26]:

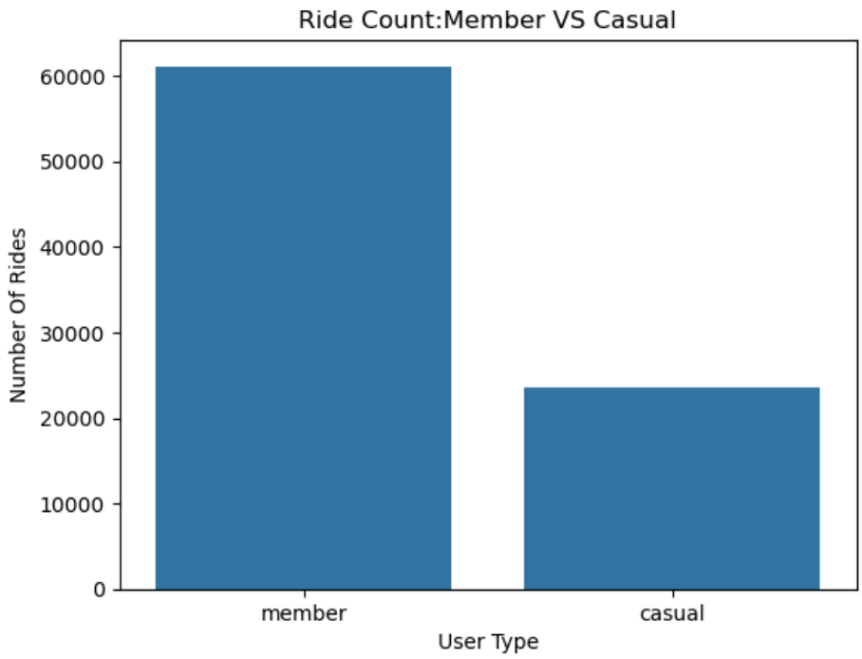
member_casual	casual	member
rideable_type		
docked_bike	23605	61112

There is only 1 byketypr ie.docked_bike

Lets Start With Visualization Now

1. User Distribution Bar Chart

In [28]:
```
sns.countplot(x='member_casual',data=df)
plt.title("Ride Count:Member VS Casual")
plt.xlabel("User Type")
plt.ylabel("Number Of Rides")
plt.show()
```

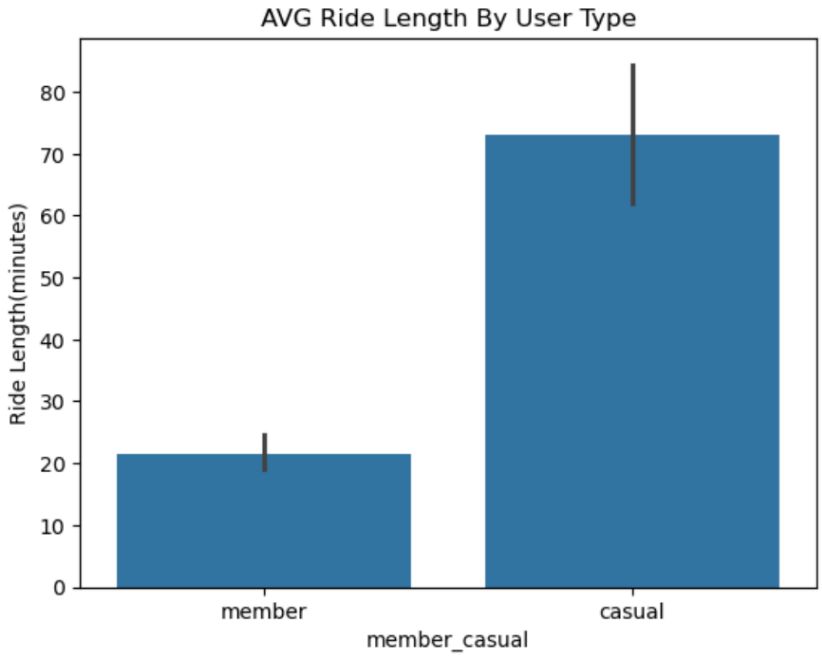


Here we can see that Members domimate the usage

2.AVG Ride Length

Figure 9 – Notebook page 5.

In [29]:
```
sns.barplot(x='member_casual',y='ride_length',data=df)
plt.title("AVG Ride Length By User Type")
plt.ylabel("Ride Length(minutes)")
plt.show()
```



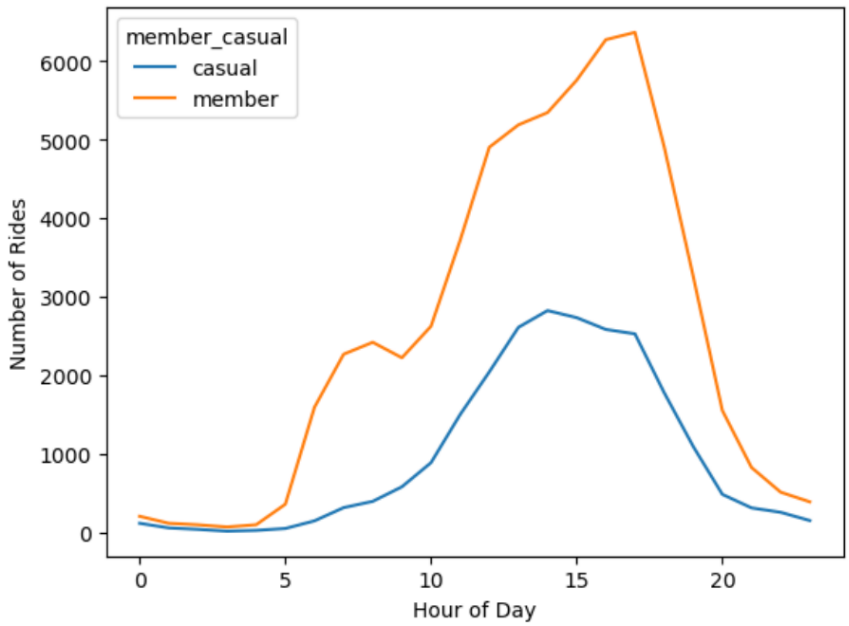
Here we can see thet casual ride longer than members

3.Hour Wise Usage

In [37]:
```
hourly=df.groupby(['hour','member_casual']).size().reset_index(name='count')

sns.lineplot(data=hourly,x='hour',y='count',hue='member_casual')
plt.title=("Hourly Usage Pattern")
plt.xlabel("Hour of Day")
plt.ylabel("Number of Rides")
plt.show()
```

Figure 10 – Notebook page 6.



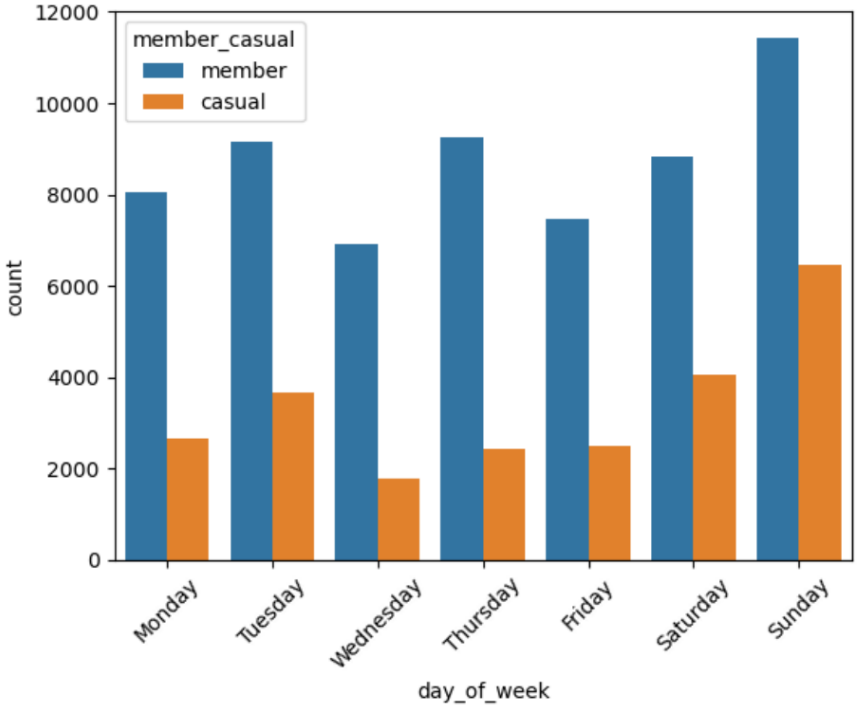
Here we can see that Member peaks (commute) Casual peaks (leisure)

4.Day-wise Usage

In [38]:
```
day_order = ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sund

sns.countplot(x='day_of_week', hue='member_casual', data=df, order=day_order)
plt.title=("Usage by Day of Week")
plt.xticks(rotation=45)
plt.show()
```

Figure 11 – Notebook page 7.



Here we can see that on sundays there are most rides taken by both casuals and mambers

```
In [ ]:
```

Figure 12 – Notebook page 8.